Heaps: Implementation and Usage

K-Way Merge / Merging k Sorted Lists (cont.)

K-Way Merge

Input: k sorted lists
Output: a single sorted list

Example (k=4)

List 1: [1, 4, 5]
List 2: [1, 3, 5]
List 3: [2, 6]
List 4: [0, 7, 8]

Resulting list: [0, 1, 1, 2, 3, 4, 4, 5, 6, 7, 8]

K-Way Merge: O(kN) solution

Here,

k = number of input lists

N = total number of elements in all input lists

Observation:

Each list has its smallest item at index 0

Idea:

Step1: Iterate over the input lists and find the smallest item.

Step2: Remove it from the input list and add to result.

Step3: Repeat step 1 and 2 until all the lists become empty!

K-Way Merge: A better solution using Heap

Idea: If we use a heap, step 1 (previous slide) can be performed in O(logk)

Thus, using a heap, K-Way Merge can be done in O(Nlogk)

K-Way Merge: A better solution using Heap

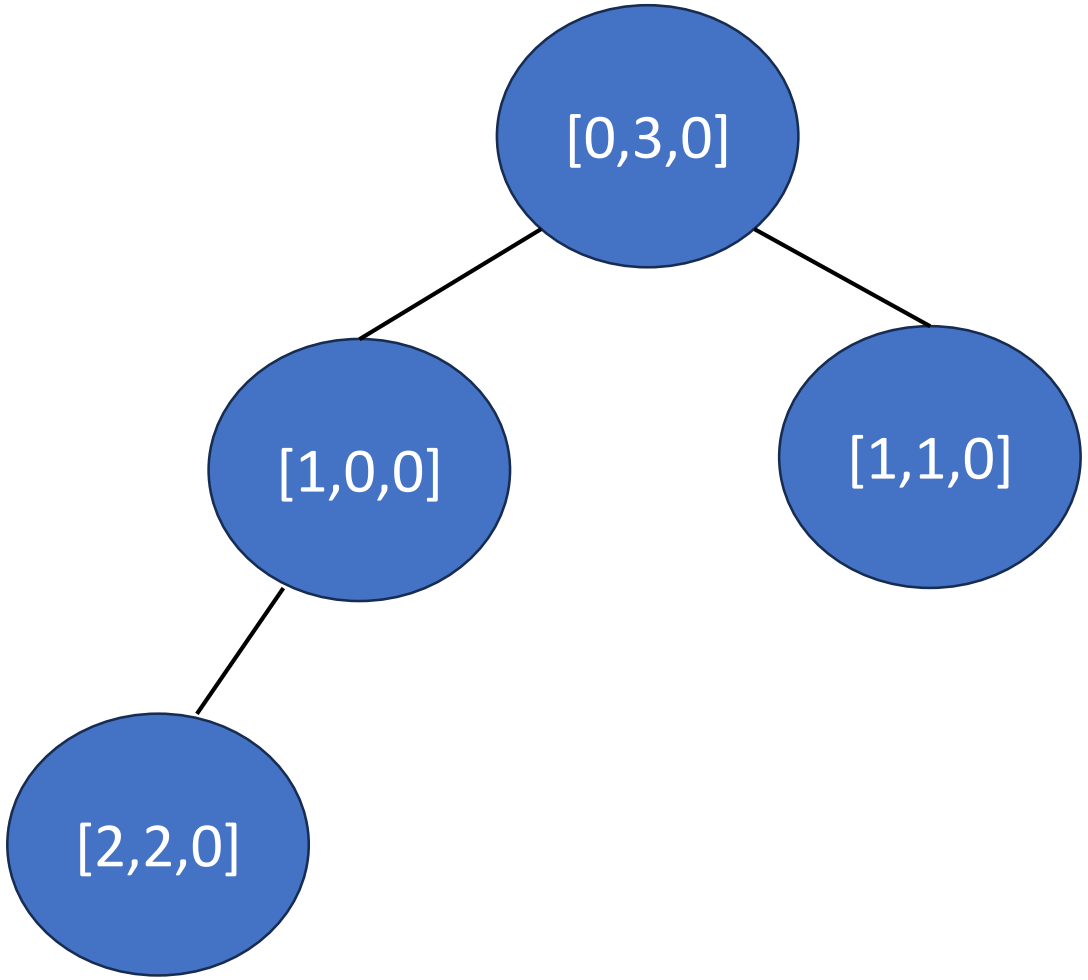
```
k = 4, N = 11

List 1: [1, 4, 5]
List 2: [1, 3, 5]
List 3: [2, 6]
List 4: [0, 7, 8]

Step 1: Build a min-heap using the numbers in the first column
Step 2: Remove the root of the heap. Add it to result.
Step 3: Insert the next available number into the heap from the
        same list from which the last number was removed.
Step 4: Repeat step 2 and 3 until the heap becomes empty.
```

K-Way Merge: A better solution using Heap

```
List 0: [1, 4, 5]
List 1: [1, 3, 5]
List 2: [2, 6]
List 3: [0, 7, 8]
```

Step 1:
Build a min-heap using the numbers in the first column

Each node of the heap:
<value, list_index, element_index>



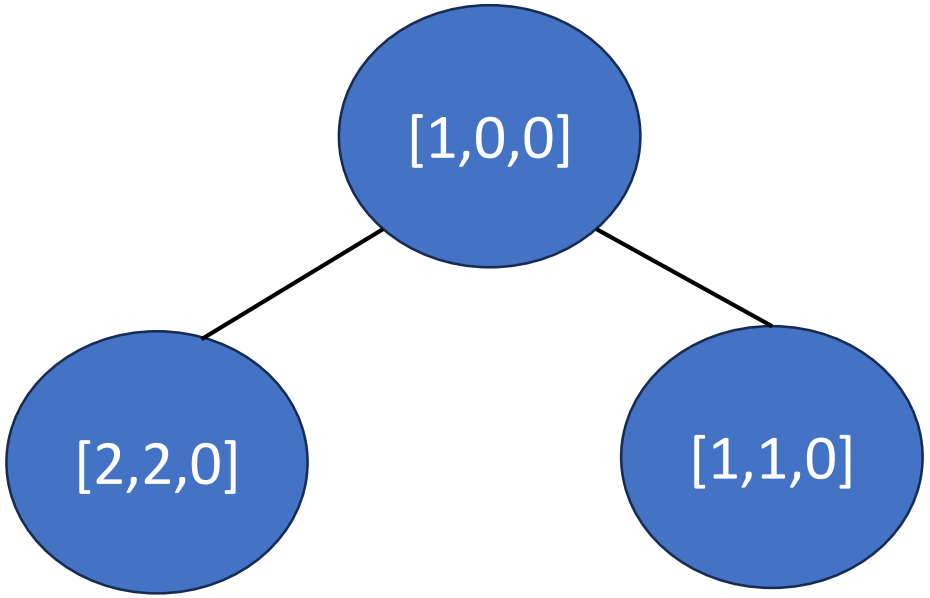
K-Way Merge: A better solution using Heap

```
List 0: [1, 4, 5]
List 1: [1, 3, 5]
List 2: [2, 6]
List 3: [0, 7, 8]
```

Step 2:

```
Remove the root of the heap.
Add it to result.
```

Res = [0]

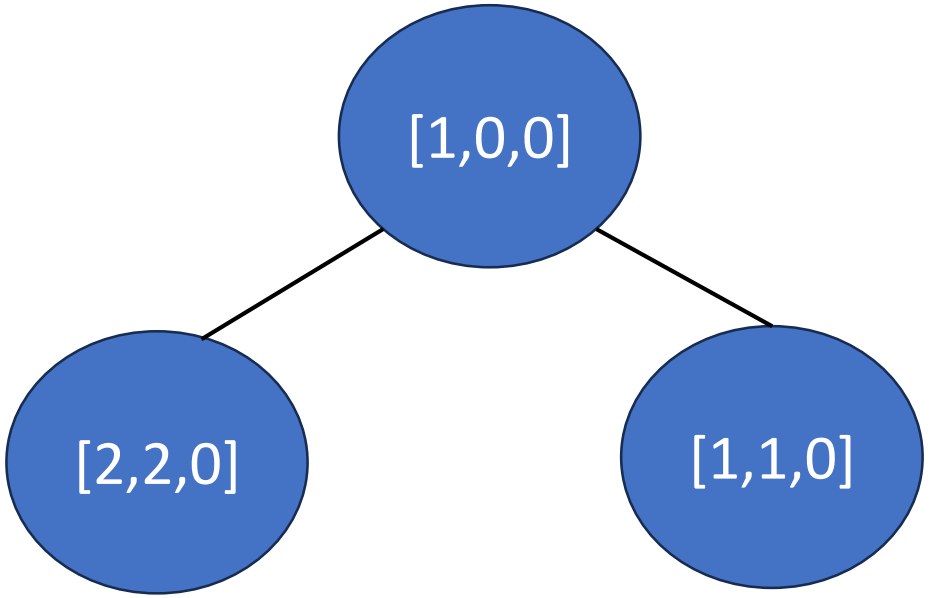


K-Way Merge: A better solution using Heap

```
List 0: [1, 4, 5]
List 1: [1, 3, 5]
List 2: [2, 6]
List 3: [0, 7, 8]
```

Step 3:
Insert the next available number into the heap from the same list

Res = [0]



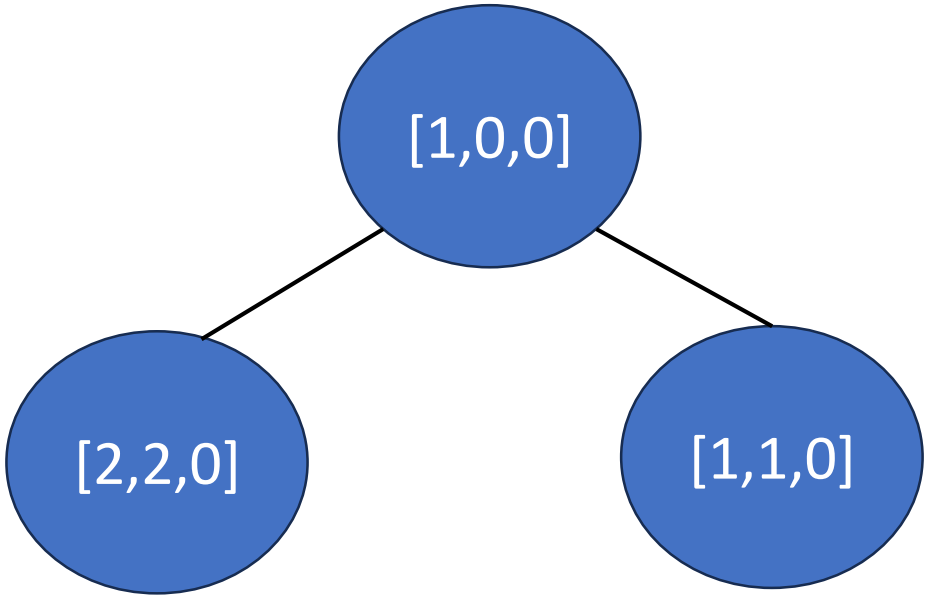
Last removed node = [0,3,0]

K-Way Merge: A better solution using Heap

```
List 0: [1, 4, 5]
List 1: [1, 3, 5]
List 2: [2, 6]
List 3: [0, 7, 8]
```

Step 3:
Insert the next available number into the heap from the same list

Res = [0]



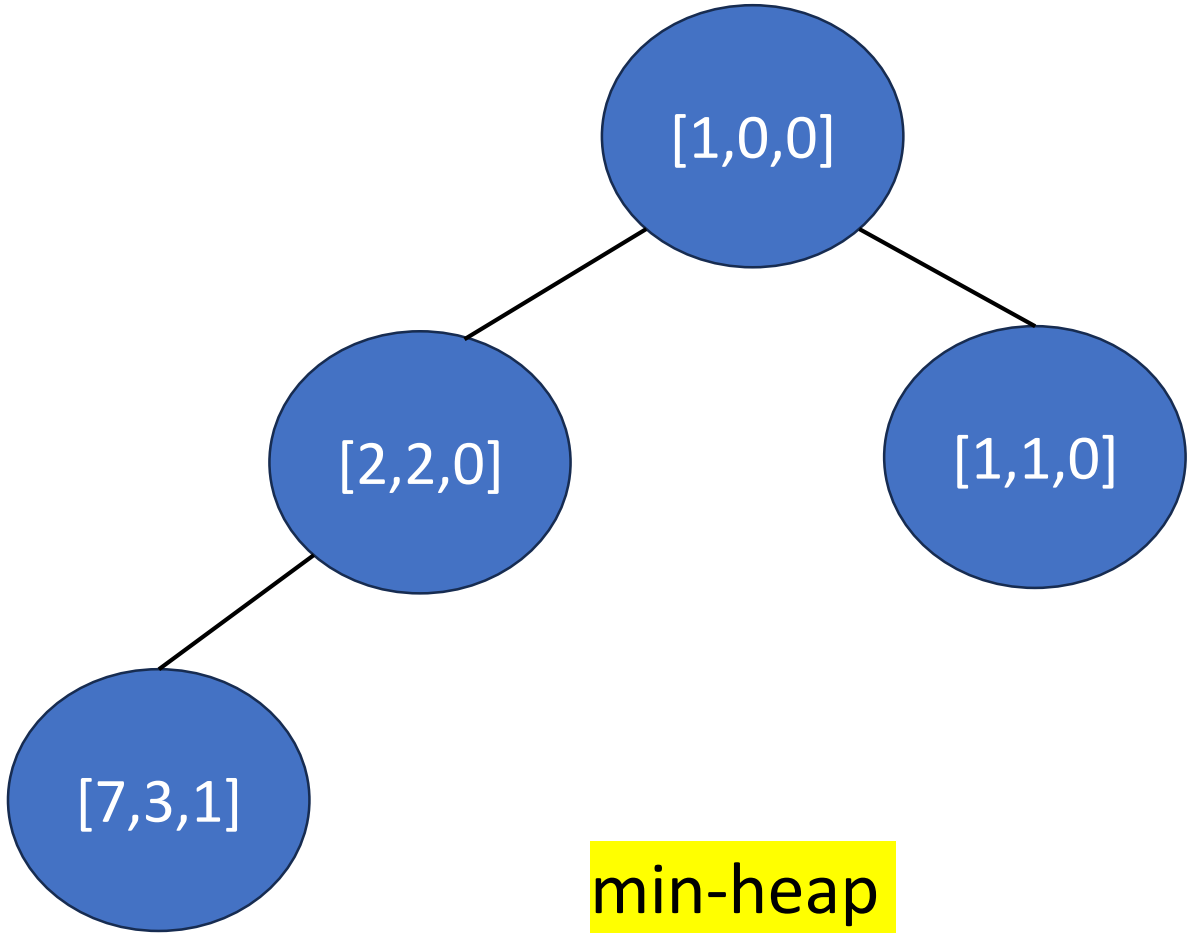
Last removed node = [0,3,0]

Next number to be inserted: 7 (element#1 from list #3)

K-Way Merge: A better solution using Heap

```
List 0: [1, 4, 5]
List 1: [1, 3, 5]
List 2: [2, 6]
List 3: [0, 7, 8]
```

Res = [0]



Repeat the same steps N times / until the min-heap becomes empty.

```python
import heapq

def k_way_merge(lists):
    # Initialize a min-heap
    min_heap = []

    # Add the first element of each list to the heap
    for i in range(len(lists)):
        sorted_list = lists[i]
        if sorted_list:  # Check if the list is not empty
            heapq.heappush(min_heap, (sorted_list[0], i, 0))  # (value, list index, element index)

    res = []

    while min_heap:
        # Get the smallest element from the heap
        value, list_idx, elem_idx = heapq.heappop(min_heap)
        res.append(value)

        # If there is a next element in the same list, add it to the heap
        if elem_idx + 1 < len(lists[list_idx]):
            next_value = lists[list_idx][elem_idx + 1]
            heapq.heappush(min_heap, (next_value, list_idx, elem_idx + 1))

    return res
```

```python
# Example usage
lists = [
    [1, 4, 5],
    [1, 3, 4],
    [2, 6],
    [0, 7, 8]
]

result = k_way_merge(lists)
print(result)
```

Median Maintenance

Median

For an odd number of elements: The middle element
For an even number of elements: The average of the two middle elements

Examples:

[1, 3, 5, 7, 9] → Median = 5
[1, 3, 5, 7] → Median = (3 + 5) / 2 = 4

Median Maintenance

Problem Statement:

Given a stream of numbers, maintain the current median efficiently as new numbers are added/removed.

Median Maintenance

Brute Force approach:

Sort the data after each insertion and find the median.

Time Complexity: O(nlogn)

Median Maintenance using Heaps

Step 1: Initialize two Heaps

Create a Max-Heap (representing for lower half) and a Min-Heap (representing upper half).

Step 2: Insert Number

Add it to Max-Heap if the num ≤ max of Max-Heap; otherwise, add to Min-Heap.

Step 3: Balance Heaps

- If Max-Heap has more than one extra element, move its root to Min-Heap.
- If Min-Heap has more elements, move its root to Max-Heap.

Step 4: Find Median

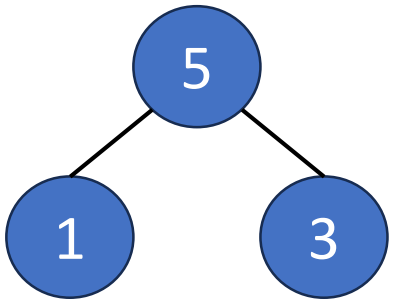
- If Max-Heap has more elements: Median = root of Max-Heap.
- If equal size: Median = (root of Max-Heap + root of Min-Heap) / 2.

O(logn) insertion time and O(1) median retrieval time.

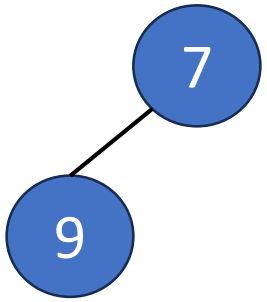
Step 1: Initialize two Heaps

Create a Max-Heap (representing for lower half) and a Min-Heap (representing upper half).

Current data = [1, 3, 5, 7, 9]



Max-heap
(lower-half of data)



Min-heap
(upper-half of data)

Median?

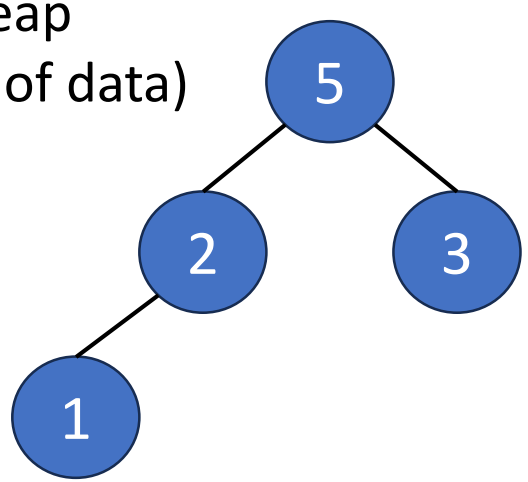
Step 2: Insert Number

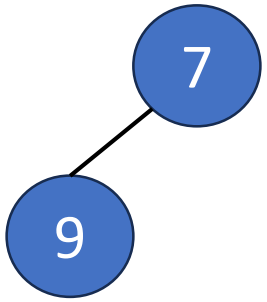
Add it to Max-Heap if the num ≤ max of Max-Heap; otherwise, add to Min-Heap.

Current data = [1, 3, 5, 7, 9]

Insert: 2

Max-heap
(lower-half of data)



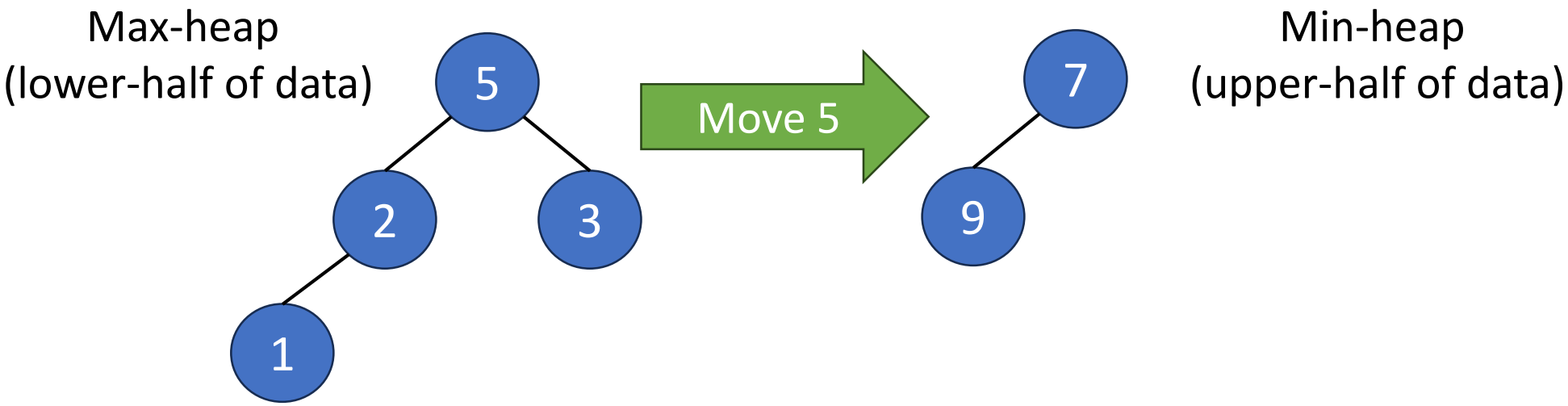


Min-heap
(upper-half of data)

Add it to Max-Heap if the num ≤ max of Max-Heap; otherwise, add to Min-Heap.

Step 3: Balance Heaps

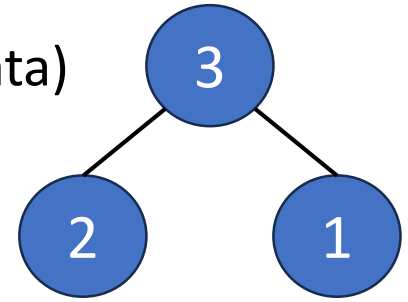
- If Max-Heap has more than one extra element, move its root to Min-Heap.
- If Min-Heap has more elements, move its root to Max-Heap.

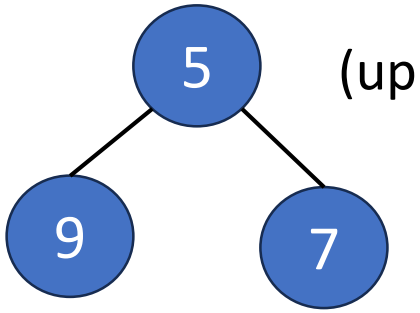


Step 3: Balance Heaps

- If Max-Heap has more than one extra element, move its root to Min-Heap.
- If Min-Heap has more elements, move its root to Max-Heap.

Max-heap
(lower-half of data)





Min-heap
(upper-half of data)

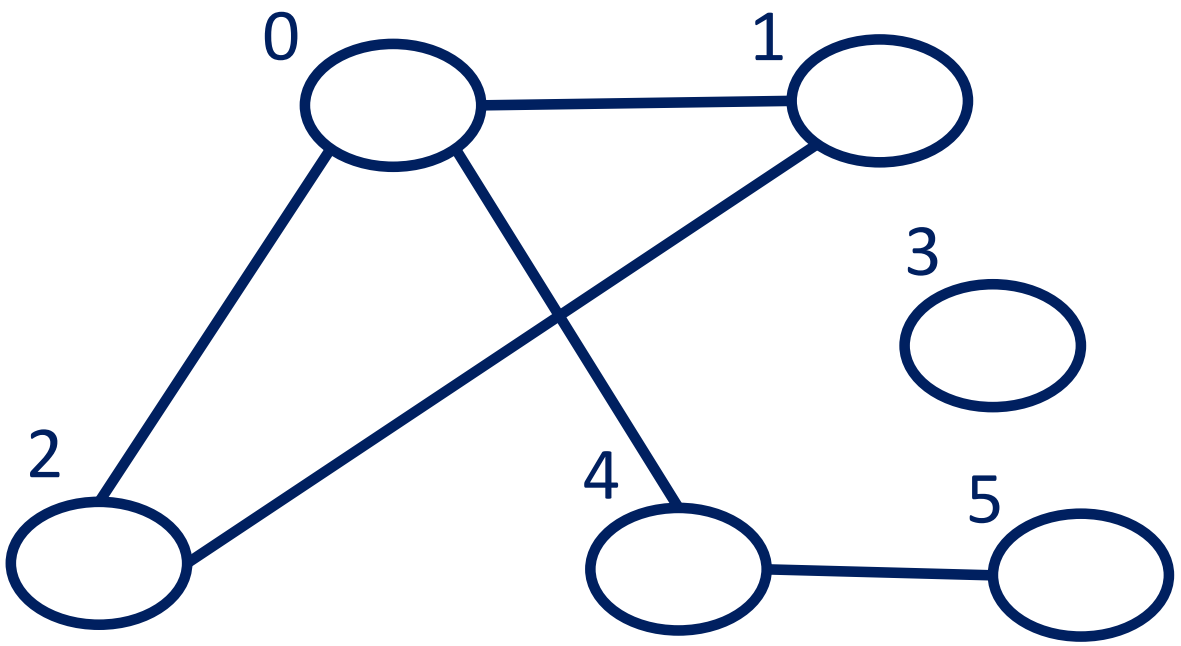
Median?

Step 4: Find Median

- If Max-Heap has more elements: Median = root of Max-Heap.
- If equal size: Median = (root of Max-Heap + root of Min-Heap) / 2.

Graphs

Graph $G = (V, E)$



A pair of sets:

- nodes (a.k.a. vertices)
  - nodes are any objects
- edges
  - edges are pairs of nodes

- Nodes:

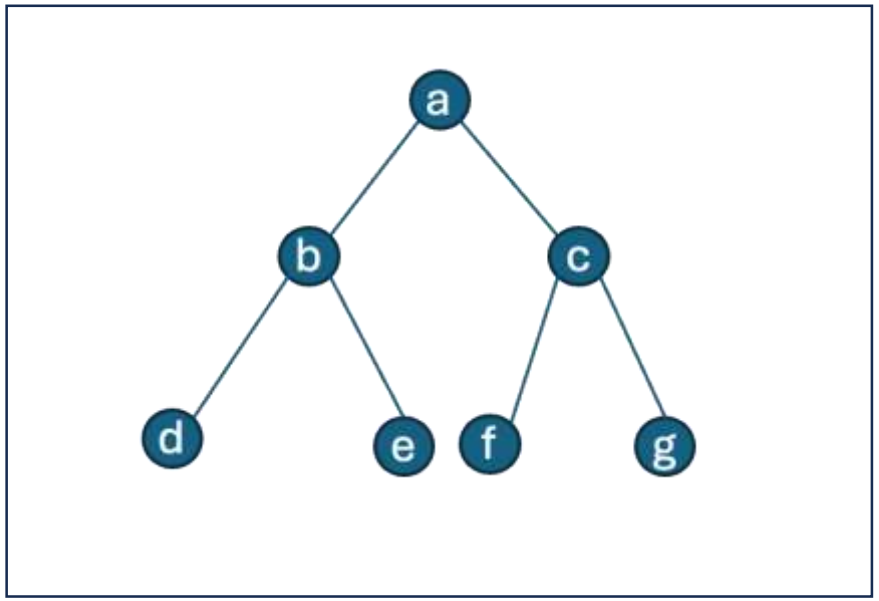
$$V = \{0,1,2,3,4,5\}$$

- Edges:

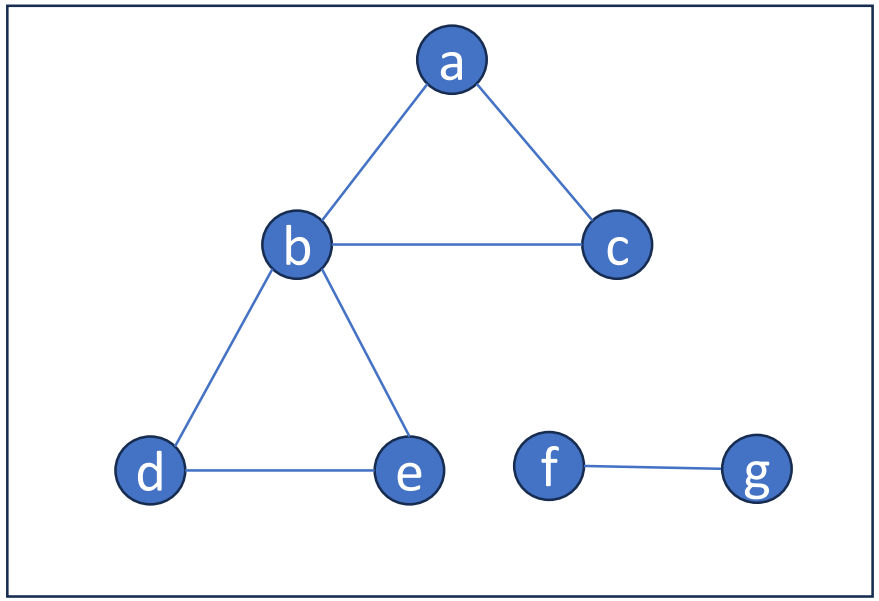
$$E = \left\{ \begin{matrix} \{0,1\}, \{0,2\}, \{0,4\}, \\ \{1,2\}, \{4,5\} \end{matrix} \right\}$$

Trees vs Graphs?

- **Cycles:** Trees don't have any cycles, while graphs may have cycles
- **Connectivity:** All the nodes of a tree are connected, which may not be the case for graphs.
- **Edge direction:** Edges of a graph can be directed/undirected
- **Application:** Suitable for different application scenarios.
    - E.g. Tree → File System, Graph → Social Network

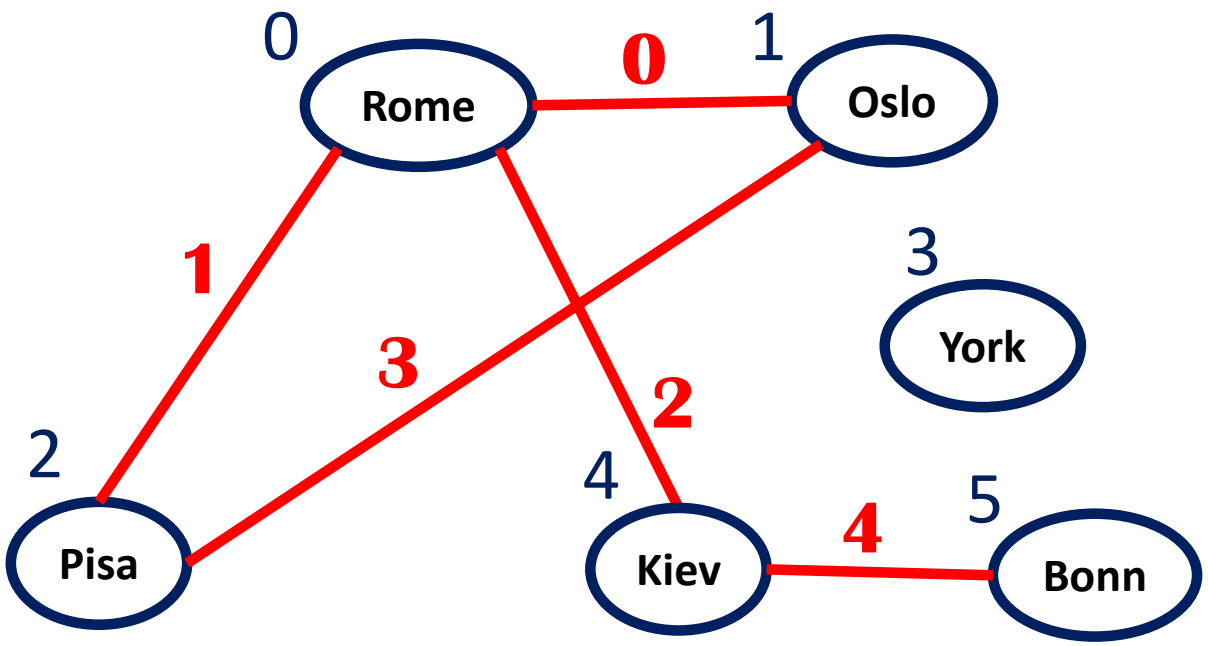


A binary tree



A graph

Graph $G = (V, E)$

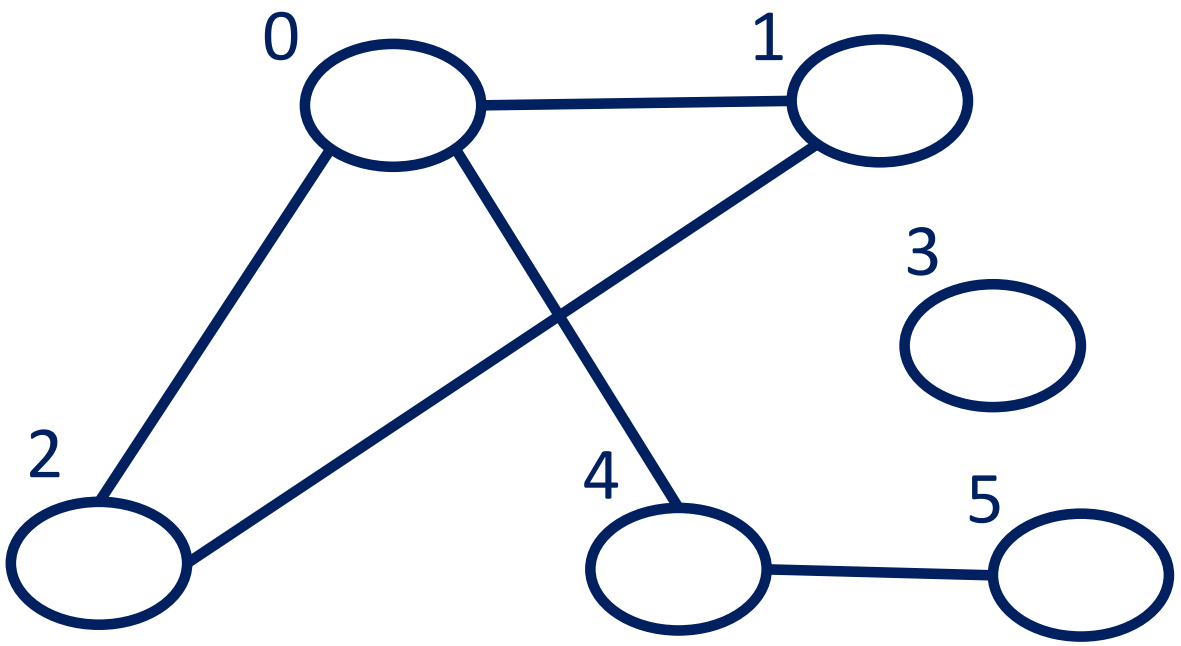


Notation for:

- nodes/vertices
  - v or v_i
- edges
  - e or e_i

- Nodes:
  $V = \{v_0, v_1, v_2, v_3, v_4, v_5\}$
- Edges:
  $E = \{e_0, e_1, e_2, e_3, e_4\}$

Graph $G = (V, E)$



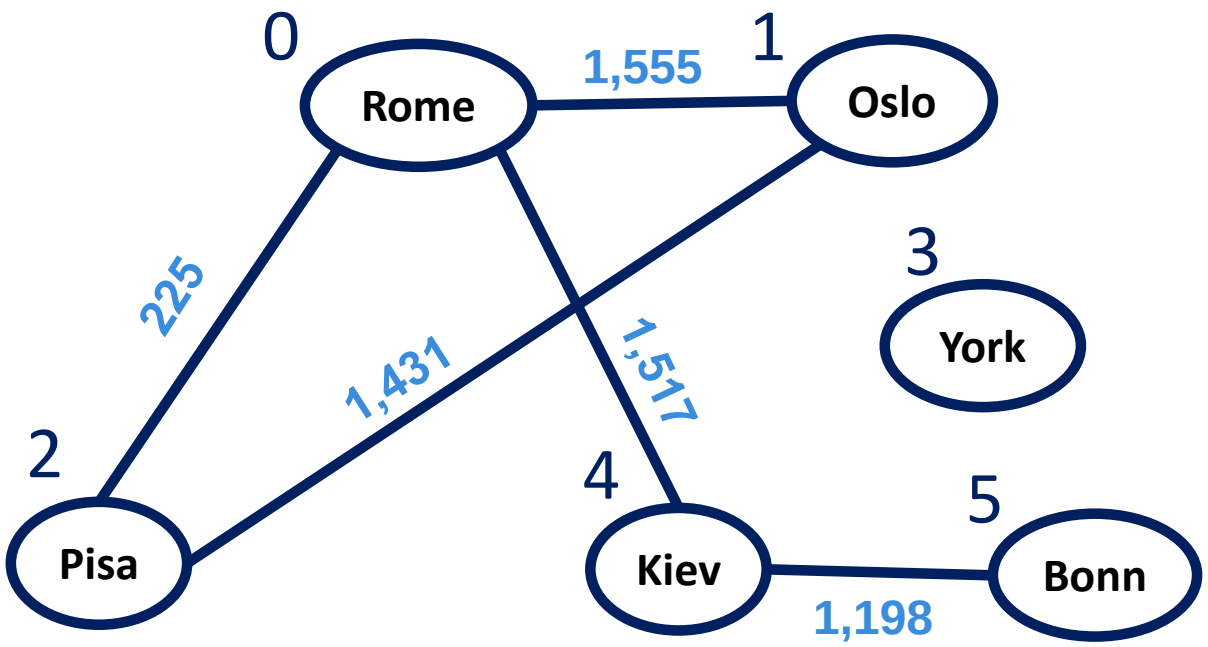
Let

- n be the number of nodes
  - i.e., $n = |V|$
- m be the number of edges
  - i.e., $m = |E|$

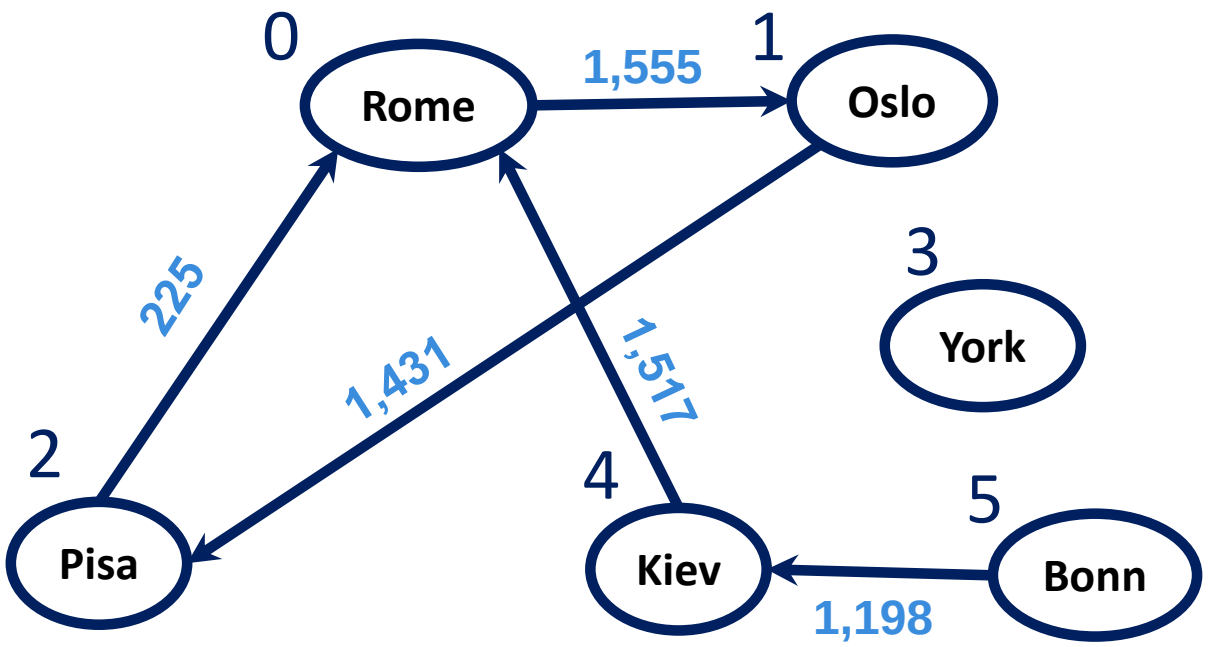
Graph $G = (V, E)$

Edges (and sometimes nodes)

- may be **weighted**



Graph $G = (V, E)$



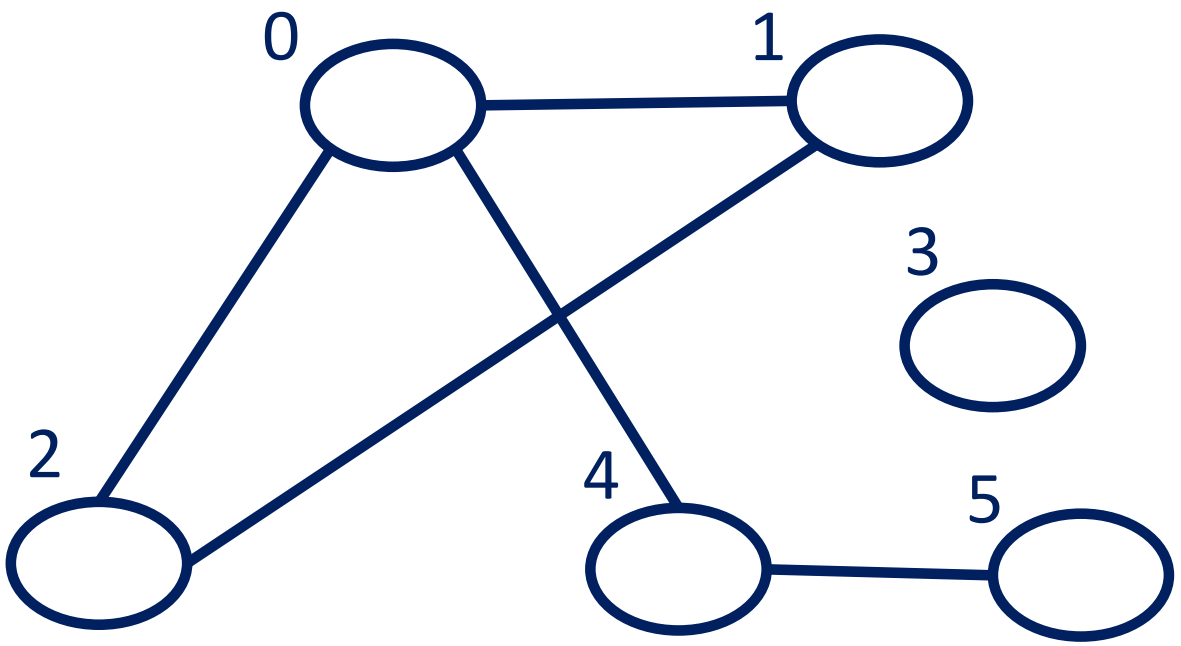
Edges (and sometimes nodes)

- may be **weighted**

Edges

- may be **directed** (a.k.a. *arcs*)

Adjacency



- Two nodes are adjacent
  if they are connected by an edge

- Two edges are adjacent*
  if they share a node

- Adjacent nodes:
  0 and 1; 0 and 4; 5 and 4

- Non-adjacent nodes:
  0 and 5; 0 and 3; 1 and 4

- Adjacent edges:
  {0,1} and {1,2};
  {0,4} and {4,5}

- Non-adjacent edges:
  {0,1} and {4,5};
  {0,2} and {4,5}

*edges incidence is more rarely used in practice the nodes incidence

Graph representation

- Graph may be represented as:
  - Matrix
    - ✓ Adjacency matrix
    - ✓ Incidence matrix
    - ✓ Laplacian matrix
    - ✓ …
  - List
    - ✓ Adjacency list
    - ✓ …
  - Others …

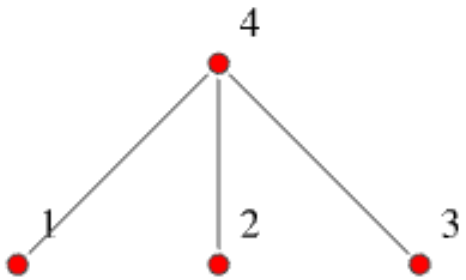
- There are *very many* different graph representations
- It depends on what kind of problem you are solving

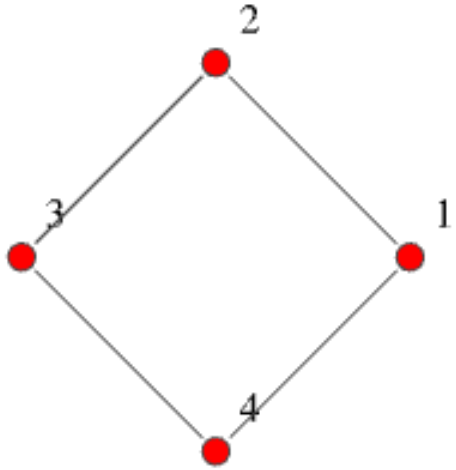
Adjacency matrix

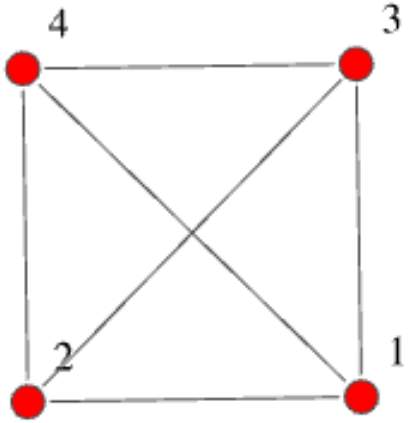
- A square Boolean* $n \times n$ matrix
- Elements on the main diagonal are zeros
- Other elements indicate if pairs of nodes are adjacent
- Often denoted as $A = [a_{ij}]_{n \times n}$
- $a_{ij} = \begin{cases} 1 & \text{if } v_i \text{ is adjacent to } v_j \\ 0 & \text{if } v_i \text{ is not adjacent to } v_j \end{cases}$

*In an unweighted graph

Adjacency matrix: examples



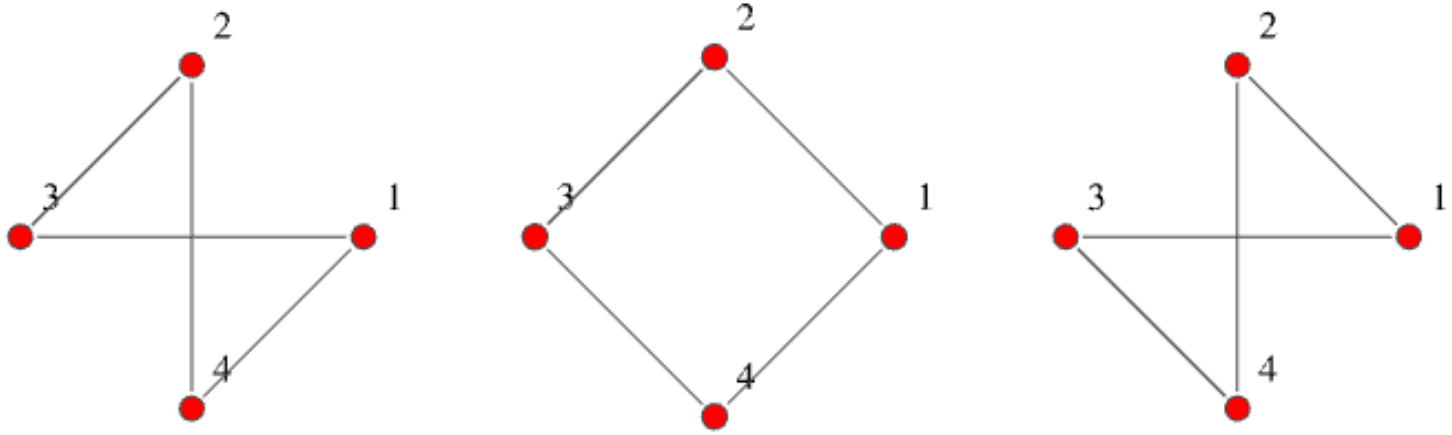




$$\begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \qquad \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \qquad \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

From https://mathworld.wolfram.com/AdjacencyMatrix.html

Adjacency matrix: examples



$$\begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix} \qquad \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \qquad \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

From https://mathworld.wolfram.com/AdjacencyMatrix.html

Main operations

- Add/remove a node
- Add/remove an edge
- Find if an edge exists
- Find neighbors
- …

Complexity

- ❑ Time

$O(n^2)$ ✓ Add a node

$O(n^2)$ ✓ Remove a node

$O(1)$ ✓ Add an edge

$O(1)$ ✓ Remove an edge

$O(1)$ ✓ Are two nodes adjacent?

$O(n)$ ✓ Find neighbors of a node

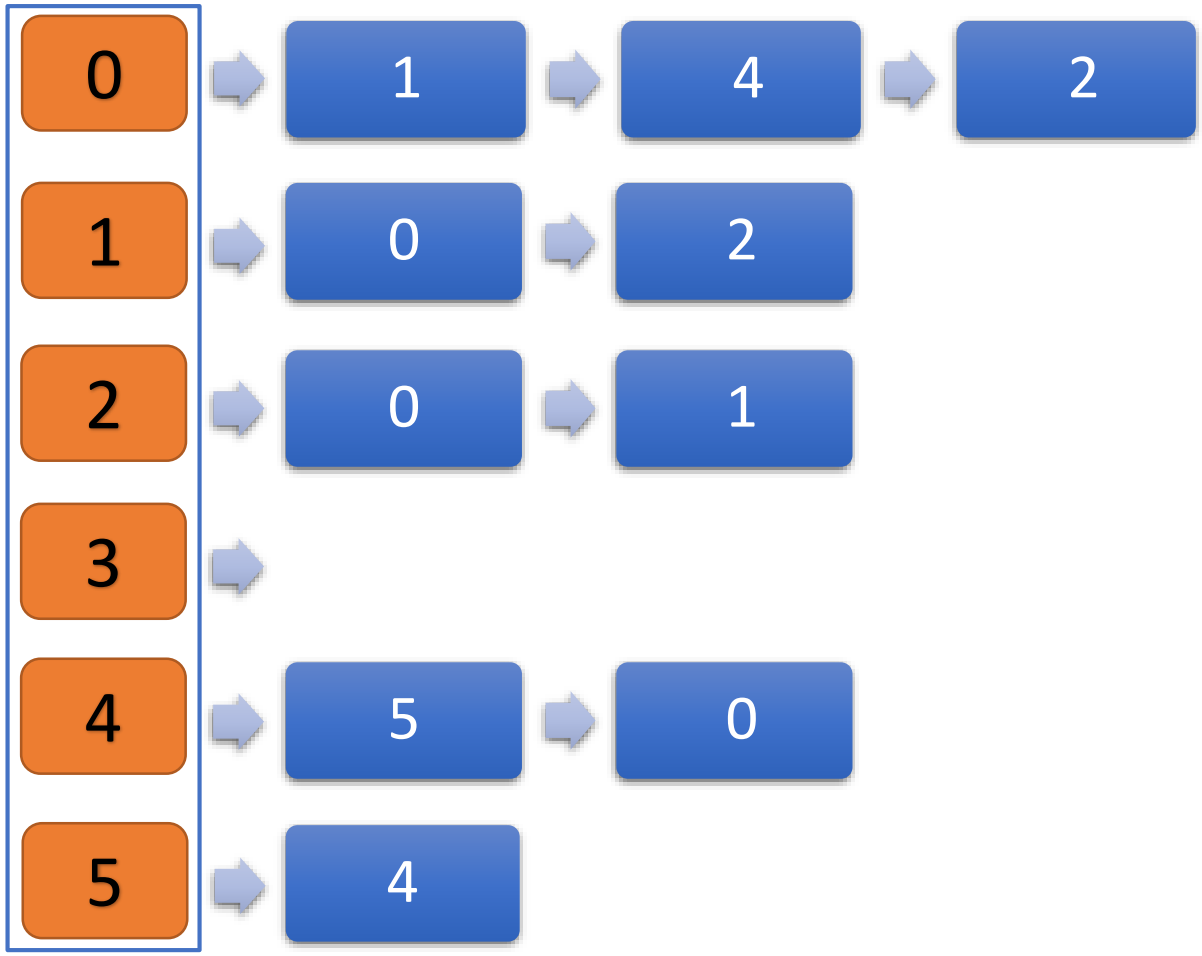
- ❑ Space

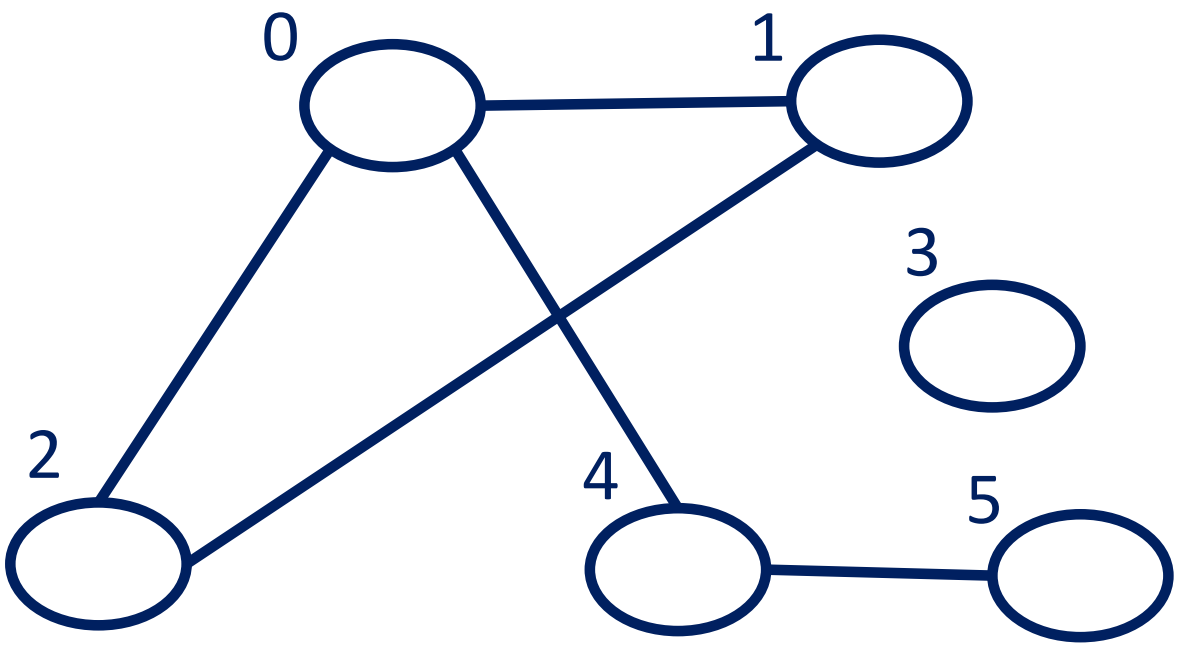
➢ $O(n^2)$

Adjacency list

- A collection (an array / a hash map) of (linked) lists / arrays
- Each list
  - ✓ corresponds to a node
  - ✓ stores a list of nodes that are adjacent to the node

Adjacency list: example





Complexity

- Time

$O(1)$	✓ Add a node
$O(m)$	✓ Remove a node
$O(1)$	✓ Add an edge
$O(n)$	✓ Remove an edge
$O(n)$	✓ Are two nodes adjacent?
$O(n)$	✓ Find neighbors of a node

- Space
  - $O(n+m)$

Time complexity: summary

	Adjacency matrix	Adjacency list
Store a whole graph	$O(n^2)$	$O(n+m)$
Add a node	$O(n^2)$	$O(1)$
Remove a node	$O(n^2)$	$O(m)$
Add an edge	$O(1)$	$O(1)$
Remove an edge	$O(1)$	$O(n)$
Are two nodes adjacent?	$O(1)$	$O(n)$
Find neighbors of a node	$O(n)$	$O(n)$
Comments	Slow to **add/remove nodes** as the matrix must be resized/copied	Slow to **remove nodes and edges** because it must find all nodes/edges

Attendance